Laboratorio 9 – Colas de prioridad

Grupo G03

Estudiante 1

Nombre: Jimmy Alejandro Mujica Delgado

Código: 202224528

Correo: j.mujicad@uniandes.edu.co

Estudiante 2

Nombre: Samuel Diaz Esteban Arias

Código: 202522112

Correo: se.diaza12@uniandes.edu.co

a) ¿Qué estructura de datos subyacente se utiliza comúnmente para implementar una cola de prioridad en Python?

Normalmente se usa un heap, binario, que es como un árbol binario casi completo pero guardado dentro de una lista. Cada padre tiene mejor prioridad que sus hijos, así que el elemento mas importante siempre queda en la raíz. En el laboratorio lo implementamos con un array_list y dejamos un None en la posición 0. Así el padre de la posición i esta en i//2 y sus hijos en 2i y 2i+1, y las cuentas son mas fáciles.

b) ¿Cuál es la diferencia entre una cola FIFO tradicional y una cola de prioridad?

En una cola FIFO los elementos salen en el mismo orden en que entraron, como la fila de un banco. En una cola de prioridad no importa quien llego primero, sino quien tiene la mejor prioridad. Por ejemplo, en nuestra MinPQ de buses salía primero la ruta con la hora de salida mas temprana, aunque hubiera sido una de las ultimas en cargarse en el archivo.

c) ¿Qué modulo proporciona Python para trabajar fácilmente con colas de prioridad?

Python tiene el modulo heapq, que trabaja sobre una lista normal y trae funciones como heappush y heappop para insertar y sacar el menor elemento. También existe queue.PriorityQueue, que hace algo parecido, pero está pensada para cuando hay varios hilos. En el curso no los usamos porque la idea era implementar la estructura nosotros mismos.

d) ¿Qué ventajas tiene el uso de una cola de prioridad sobre una lista ordenada manualmente?

La principal ventaja es la eficiencia. En una lista ordenada, cada vez que se agrega un elemento hay que buscar su lugar y mover los demás, lo que puede ser O(n). En el heap solo se hace uso de las funciones swim() o sink(), que recorren la altura del árbol

O(log(n)). Además, el código queda mas ordenado, porque ya tenemos funciones como insert, remove y get_first_priority y no hay que preocuparse por ordenar a mano.

e) Si dos elementos tienen la misma prioridad, ¿Cómo decide la cola cual atender primero?

El heap no garantiza un orden entre los elementos con la misma prioridad: depende de cómo vayan quedando después de los intercambios en swim y sink. Esto lo vimos durante la práctica del laboratorio, porque varias rutas tenían como prioridad 0500 y fueron saliendo en un orden que no seguía ningún patrón. Si se quisiera un orden fijo, habría que agregar un segundo criterio de comparación, por ejemplo, el orden de llegada o el numero de la ruta.

f) ¿Qué situaciones del mundo real se pueden modelar con colas de prioridad? Mencione al menos dos

- Urgencia de un hospital: no se atiende por orden de llegada sino según la gravedad de un paciente
- Procesos de un computador: el sistema operativo decide que programa ejecutar primero según su prioridad

g) En un sistema de atención médica, ¿Cómo se puede usar una cola de prioridad para organizar a los pacientes?

Cuando llega un paciente, se le hace triage y se le asigna un nivel de prioridad según que tan grave esta. Luego se inserta en la cola con insert. Cuando un medico queda libre, se hace remove y sale el paciente mas grave. Si un paciente empeora mientras espera, se podría usar improve_priority para subirlo en la cola sin sacarlo y volverlo a meter.

h) ¿Cómo afectaría al rendimiento usar una lista simple en lugar de una estructura especializada como heap_pq para manejar prioridades?

Con una lista simple sin ordenar, insertar es rápido, pero para encontrar el elemento con mayor prioridad hay que recorrer toda la lista O(n). Si la lista se mantiene ordenada, sacar el primero es fácil, pero cada inserción cuesta O(n). Con pocos datos no se nota, pero con los casi 14000 registros de buses, o con mas, la diferencia si seria grande frente al heap.

i) ¿Qué complejidad temporal tiene la inserción y eliminación/atención en una cola de prioridad basada en heap?

Tanto al insertar como eliminar son O(log n). Al insertar, el elemento se pone al final y sube con swim; al eliminar, el ultimo pasa a la raíz y baja con sink. En los dos casos, como mucho se recorre la altura del árbol, que es log n. Consultar el primer elemento sin sacarlo (get_first_priority) es O(1), porque siempre está en la posición 1.